# Project 1 Report: A Flex Lexer for Anka

BIL 463 Compiler Design · Mert Eldemir (25120205086), working alone · Path A: C + Flex · 2026-10-16

## 1 · Language design

*Why these keywords and operators?*

Anka is the language I will keep using for the whole semester (the parser, type checker and code generator come next), so I only added tokens that a later phase really needs:

- **Functions:** `fn`, `return`, and the arrow `->` for the return type.
- **Variables:** `let x: int = 5;` The type comes after the name, like in Rust. A declaration always starts with `let`, so the parser never has to guess whether a word is a type or a variable.
- **Control flow:** `if`/`else`, `while`, `for i in 0..10`, `break`, `continue`. **Types:** only `int`, `float`, `bool` (with `true`/`false`), so type checking stays simple.
- **`print` is a keyword** because Anka has no string type. A string can only appear inside `print(...)`, so I won't have to manage string memory in the code generator.

The operators are the usual C ones (arithmetic, comparison, `&&`, `||`, `!`) plus `**` for power, `+=`-style compound assignment, `->` and `..`. In total there are 16 keywords and 22 operators, and 13 of the operators are two characters long.

*How are multi-character operators handled (`=` vs `==` vs `<=`)?*

I didn't write any special code for this. Every operator is simply its own rule in `lexer.l`: `"="` is one rule, `"=="` is another, the same for `"<"` and `"<="`. Flex then follows **maximal munch**: it always takes the *longest* piece of input that matches a rule. On `==` it could stop after the first `=`, but it keeps reading, sees that `==` also matches, and returns `EQ`. Three results from my tests:

- `a-->b` → `IDENT MINUS ARROW IDENT`. There is no `--` operator, so the first `-` is `MINUS` and then `->` follows.
- `x**=2` → `IDENT POW ASSIGN INT`, because Anka has no `**=` operator.
- `0..5` → `INT DOTDOT INT`. It does not become the float `0.`, because in Anka a float needs a digit after the dot.

*Walk-through of the input `ifx`*

| Step | What happens | Flex remembers |
|---|---|---|
| read `i` | Only the identifier rule matches so far (it could still become `if`, `in` or `int`). | 1 char = `IDENT` |
| read `f` | `if` matches **two** rules: the keyword and the identifier. Same length, so the rule written first in `lexer.l` wins. | 2 chars = `IF` |
| read `x` | `ifx` is not a keyword, only the identifier rule still matches. | 3 chars = `IDENT` |
| space / end | Nothing can continue, so Flex returns the last match it remembered. | `IDENT 'ifx'` |

So the longest match wins first (3 characters beat 2), and the order in the file only decides a tie. That's why the keywords are written *before* the identifier rule in `lexer.l`; otherwise `if` would also become an `IDENT`.

## 2 · Testing and bugs

There are 8 test files in `tests/` with 34 numbered cases (C1–C34). Each file's first comment lists its cases, and `output/` has the expected output. `make test` runs the lexer on every file and compares: **all 8 pass**. The cases cover valid programs (C1–C3, and C34, a small program I wrote myself at the end to try the lexer), maximal munch and keyword priority (C4–C12), numbers (C13–C19), strings and comments (C20–C26, C32), error recovery (C27–C30), a comment that is never closed (C31) and Windows line endings (C33).

### Three bugs

1. **A warning in code I didn't write.** Running `make` printed a `-Wsign-compare` warning (a signed number compared with an unsigned one) inside `lex.yy.c`. That file is generated by Flex, so I can't edit it. *Fix:* compile `lex.yy.c` separately with only that warning turned off. My own `main.c` still gets every warning, and the project now builds with no warnings on both clang and gcc.
2. **A backslash at the end of an unclosed string (test C32).** For `print("abc\` and then a new line I expected one error ("unterminated string") but also got "unexpected character '\'". *Why:* in my string regex an escape is `\\.`, a backslash plus one more character, and in Flex `.` never matches a newline. So the match stopped before the lone `\`, and it went to the catch-all error rule. *Fix:* the unclosed-string rule now allows one optional backslash at the end (`\\?`), so the `\` becomes part of the same error.
3. **Windows line endings (test C33).** Windows ends a line with two characters, `\r\n`, not just `\n`. In such a file, the error for an unclosed string contained the invisible `\r`, which moves the cursor back to the start of the line, so the terminal output got scrambled. *Why:* my string regex excluded `\n` but forgot `\r`. *Fix:* exclude both (`[^"\\\r\n]`). I also added a `.gitattributes` rule so git doesn't convert this test file's line endings.

For bugs 2 and 3, I built the old version of the lexer and checked that the new tests really fail on it, so the tests would have caught these bugs.

### Edge cases and surprises

- `123abc` is **not** an error for the lexer: it becomes `INT_LIT` and then `IDENT`. Rejecting it is the parser's job.
- `007` is an error but `007.5` is a valid float, like in Python 3 (C reads `007` as octal; floats don't have that problem).
- A Turkish letter like ğ outside a string gives **two** errors, not one. It takes two bytes in UTF-8, and my lexer reads bytes.
- `lex.yy.c` says it has 62 rules. Counting my rules to match that, I found my spec said 15 keywords instead of 16.

## 3 · AI usage and BPE comparison

### AI usage

**Tool:** Claude Code (Anthropic, Claude Opus 5.5) in my terminal, inside the project folder. No other AI tools. I used it to:

- plan the work and keep notes between sessions;
- design the language: it asked me multiple-choice questions and I picked every feature;
- write the first versions of the spec, the lexer, the tests and the BPE script;
- write the first draft of this report and the SPEC from my notes; I then had it simplify the wording and checked every sentence against the code and test results;
- explain every file to me in plain language, including a walkthrough page I studied before submitting.

My rules for it were: explain every change, never run git commands that change history (I made every commit myself), and write down each mistake it made in `docs/AI_USAGE.md`. Part A I did by hand; there the AI only acted as a tutor and gave me similar practice problems.

**Three of my prompts, verbatim:**

> *"let's pick it together, ask me questions and give me the interesting options" (designing the language)*

> *"yes, start the test suite with the 6-file plan"*

> *"before that I want from you the complete in human way understandable explanations about what you have done on each file, what is the purpose, what is the code that you've written serve for, imagine that I'm someone who is new at the compiler design so I want to truly understand what was done"*

**What the AI got wrong, and how it was fixed:**

- Bugs 2 and 3 from Section 2 came from its string rules. Its own edge-case tests found them, and both now have tests.
- The first version of the spec said 15 keywords and 10 two-character operators. The real numbers are 16 and 13. I caught this through the 62 rules in `lex.yy.c`.
- Its first float regex accepted `007.5` while `007` was an error, and it didn't notice that the two rules disagree. I decided to keep this behaviour on purpose, and the spec now explains why.
- Its first end-of-session script ran `git commit` by itself, against my rule. It was changed to only print the commands, and git writes are now blocked in the tool's settings.

### BPE comparison

I used tiktoken with the GPT-4 tokenizer (`cl100k_base`, tiktoken 0.14.0). The script is `bpe/bpe_compare.py`, and the full output is in `bpe/results.md` (`make bpe` creates it again). In the table, ␣ stands for a space.

| Input | My lexer | BPE (GPT-4) |
|---|---|---|
| All 8 test files (5,503 characters) | 512 tokens (+ 27 errors) | 1,810 tokens |
| Same code without comments or extra spaces | 512 | 672 |
| `let x = 42;` | 5: `LET IDENT ASSIGN INT_LIT SEMI` | 6: `let · ␣x · ␣= · ␣ · 42 · ;` |
| `3.14159 · 1000` | `1 · 1` | 4: `3 · . · 141 · 59` · 2: `100 · 0` |
| `ifx` | 1: `IDENT(ifx)` | 2: `if · x` |
| x@y | `IDENT`, error `@`, `IDENT` | 3: `x · @ · y` |
| Turkish vs English sentence | (not Anka) | 14 tokens for 3 words vs 4 tokens for 4 words |

1. **How many tokens, and why the difference?** BPE gives about 3.5 times more tokens (1,810 vs 512). The main reason is that BPE also turns comments, spaces and newlines into tokens, while my lexer throws them away. If I give BPE only the code (no comments, single spaces), the gap shrinks to 672 vs 512, about 1.3 times. The rest is because BPE cuts long numbers and rare words into pieces. Sometimes it's the other way around: `-->` is one BPE token, but my lexer gives `MINUS ARROW`.
2. **42:** BPE keeps `42` as one token, but the space before it becomes a separate token. GPT-4 splits numbers into groups of at most 3 digits (`1000` → `100 · 0`). *Surprise:* the lecture slide shows `' 4','2'`, but none of the tokenizers I tried (gpt2, cl100k, o200k) does that; GPT-2 gives `' 42'` as one token. My lexer always makes one `INT_LIT`, however long the number is, because the parser needs the number's value, not pieces of it.
3. **Error characters like `@`:** my lexer prints `unexpected character '@'` with the line number, skips it and keeps going; the program is invalid. BPE has no idea of an error: `@` is just another token in its vocabulary. Unknown characters are split into raw bytes (the hieroglyph 𓂀 becomes 4 byte tokens), so BPE can tokenize anything.
4. **Which is better for a parser, and for a neural network?** For a **parser**, my lexer: every token has a category (`IF`, `IDENT`, `INT_LIT`), keywords and identifiers are always split correctly, and errors are reported. BPE splits `ifx` into `if · x`, so a parser would see a keyword that isn't really there. For a **neural network**, BPE: a fixed vocabulary (about 100k tokens), it never fails on any text, and common strings become one token, so inputs stay short. The network learns the meaning itself.
5. **Why does the Turkish sentence need more tokens?** `"Derleyici belirteçleri ayrıştırır"` takes 14 tokens for 3 words (about 4.7 per word), while `"The compiler parses tokens"` takes 4 tokens for 4 words. BPE learned its merges mostly from English text, so common English words are single tokens. Turkish builds words by adding suffixes (`belirteç + leri`, `ayrıştır + ır`), so a full word is rare in the training data and gets split. Letters like `ı ş ç` take 2 bytes in UTF-8 and get merged less often; `ç`, for example, ends up as a token on its own.